

## Übungsblatt 3

### Ü 3.1 Kleinbuchstaben für `char [ ]`

- a) Arrays vom Typ `char` werden in C zur Darstellung von nicht numerischen Typen, beispielsweise Buchstaben, verwendet. Erstellen Sie ein C-Programm, das eine strukturierte ASCII-Tabelle erstellt und ausgibt, die sowohl jeden `char` als auch sein numerisches Pendant (`int`) enthält (z.B. 97 für 'a').
- b) Schreiben Sie ein Programm `getlower.c`, das eine beliebige Zeichenkette einliest, die einzelnen Zeichen in Kleinbuchstaben umwandelt und diese ausgibt. Nur Großbuchstaben des Alphabets sollen bearbeitet werden. Ein Beispiel wäre:  
`KlagenfuRT AM W0erther$€€ -> klagenfurt am w0erther$€€`

### Ü 3.2 Resource Limits

Eine der Hauptaufgaben eines Betriebssystems ist das Management von Systemressourcen und deren Zuteilung an im System laufende Prozesse. Mit dem Kommando `ulimit` kann auf die Vergabe der Systemressourcen eingegriffen werden. Finden Sie heraus, wie dieses Kommando funktioniert (`man bash` bzw. `ulimit --help`) und lösen Sie folgende Aufgaben:

- a) Schreiben Sie ein C-Programm `endless.c`, welches eine Endlosschleife ausführt. Zeigen Sie, wie man mit `ulimit` die Ausführung von `endless` auf 4 Sekunden beschränken kann.
- b) Sehen Sie sich das Programm `recursive.c` an. Warum stürzt es ab? Gibt es eine Möglichkeit mit `ulimit` oder über den Compiler-Aufruf, das Programm doch erfolgreich auszuführen?

### Ü 3.3 Memory Management

- a. Schreiben Sie ein C-Programm `leak.c`, welches in einer Endlosschleife einen Integer-Pointer anlegt und Speicherplatz mit `malloc` allokiert. Führen Sie das Programm aus und beobachten Sie Ihre Arbeitsspeicherauslastung (`top` auf Linux oder Task Manager in Windows). Das Programm stürzt eventuell ab. Wie kann dies verhindert werden? Beachten Sie mindestens zwei Lösungsansätze: Error-Handling und Speicherfreigabe. Implementieren Sie mindestens einen davon in `leak.c`.
- b. Schreiben Sie weiters ein Programm namens `biggest.c`, welches die maximale Größe eines Integer-Arrays herausfindet, das zu dieser Zeit allokiert werden kann.

### Ü 3.4 Linux verstehen 2

- 1) Wie werden Dateitypen in Linux dargestellt?
- 2) Der Befehl `ls` zeigt die Inhalte eines Ordners an; wie kann man die Ausgabe anpassen, sodass nur der Monat der Dateien ersichtlich wird? Bsp:

```
Dec .
Dec ..
Dec .bash_history
Mar .bash_logout
Mar .bashrc
Dec .local
Mar .profile
Jan .ssh
```

- 3) Für Sicherheitssysteme spielen Zufallszahlen eine große Rolle. Finden Sie einen Weg, um eine 1-GB-große Datei mit Zufallszahlen in Linux zu generieren.

### Ü 3.5 Function Pointer

Implementieren Sie die Funktion `count_if()`, der folgende Daten als Parameter übergeben werden: (1) ein Integer-Array, (2) die Größe des Arrays und (3) ein Function Pointer `IntFunction`:

```
unsigned count_if(int array[], unsigned size, IntFunction func)
```

Deklariert Sie per `typedef` den **Function Pointer** `IntFunction`, der einer Funktion mit einem Integer-Wert als Übergabeparameter und einem Boolean als Rückgabewert entspricht. Die beiden folgenden Hilfsfunktionen entsprechen diesem Function Pointer:

```
bool isEven(int value)
bool isGreaterThanPrevious(int value)
```

Die Funktion `count_if()` soll die übergebene Funktion `func` auf jedes Array-Element anwenden.

- 1) Implementieren Sie `isEven()` und `isGreaterThanPrevious()`.
- 2) Implementieren Sie `count_if` nach der obigen Signatur. Vergessen Sie nicht, den Funktionspointer zu definieren. [ `typedef bool (*IntFunction)(int);` ]
- 3) Rufen Sie `count_if` mit beiden Hilfsfunktionen auf, für das folgende Array:

```
int values[10] = {55, 12, 98, 55, 15, 8, 52, 71, 12, 11};
```

#### Erwartetes Ergebnis:

```
isEven: 5      isGreaterThanPrevious: 4
```